

iRDRC: Deep Understanding — Sections I–III

Based on: Hieu, Hoang, Luong, Niyato

“An Intelligent Real-Time Dual-Functional Radar-Communication System
for Automotive Vehicles”

IEEE Wireless Communications Letters, Vol. 9, No. 12, December 2020

August 23, 2026

Abstract

A detailed walkthrough of the paper’s first three sections (Introduction, System Model, Problem Formulation) — the foundational concepts needed before understanding the DQN algorithm and experimental results. Written for self-study and BTP report context.

Contents

1	Motivation: Why DFRC and Why Adaptive Mode Selection?	3
1.1	The Problem Space: Autonomous Vehicles Need Multiple Capabilities . .	3
1.2	Dual-Functional Radar-Communication (DFRC): A Hardware Innovation	3
1.3	Why Existing Approaches (Fixed Scheduling) Fail	4
1.4	The Motivation for Learned, Adaptive Policies	4
2	System Model: States, Modes, Metrics	4
2.1	The Data Queue and Communication Mode	5
2.2	The Radar Mode and Unexpected Events	5
2.3	Four Environmental Factors and Their States	6
2.4	The Dual Performance Metrics and the Tradeoff	6
3	Environment Model and Event Probability (Equation 1)	7
3.1	Bayesian Combination of Per-Factor Probabilities	7
3.2	Concrete Example: Event Probability Calculation	8
3.3	Independence Assumption	8

4	Problem Formulation: The Markov Decision Process (MDP)	9
4.1	The State Space S (Equation 2)	9
4.2	The Action Space A	9
4.3	The Reward Function (Equation 3)	10
4.3.1	Case 1: Communication mode ($a = 0$), no unexpected event ($\neg\oplus$)	10
4.3.2	Case 2: Communication mode ($a = 0$), but an unexpected event occurs ($\oplus$)	10
4.3.3	Case 3: Radar mode ($a = 1$), no unexpected event ($\neg\oplus$)	10
4.3.4	Case 4: Radar mode ($a = 1$), and an unexpected event occurs ($\oplus$)	10
4.3.5	Summary Table	11
4.4	The Objective: Discounted Return and Optimal Policy (Equation 4) . . .	11
4.5	Why This Is an MDP (and Why Reinforcement Learning Is Needed) . . .	12
5	Bridge to the Next Sections (Not Covered Here)	12
6	Modeling Gaps and Questions for Implementation	13

1 Motivation: Why DFRC and Why Adaptive Mode Selection?

Paper Reference: Section I (Introduction)

1.1 The Problem Space: Autonomous Vehicles Need Multiple Capabilities

An autonomous vehicle (AV) must do two things simultaneously:

1. **Sense the environment safely** — use onboard radar to detect unexpected obstacles (occluded by other vehicles, weather-obscured, outside camera range) that could cause collisions.
2. **Communicate data** — transmit road-state information, sensor data, or video streams to roadside infrastructure (Base Stations, edge computing systems) via V2I (Vehicle-to-Infrastructure) links for centralized traffic management and cooperative driving.

Historically, these two functions required separate hardware: a radar system plus separate cellular/WiFi radios, each with its own antenna, spectrum allocation, and power budget.

1.2 Dual-Functional Radar-Communication (DFRC): A Hardware Innovation

DFRC consolidates both functions onto a **single shared radio device**. The same antenna, spectrum, and power supply do both jobs — but not simultaneously. At each discrete time step, the AV chooses one mode:

- **Mode $a = 0$ (Communication):** transmit queued data packets to base stations.
- **Mode $a = 1$ (Radar):** transmit a radar pulse and listen for reflections to detect obstacles.

Advantage: spectrum efficiency and lower cost. **Challenge:** mode selection becomes a resource-allocation problem. Use radar too much $\rightarrow$ fewer data packets get sent, reducing throughput. Use communication too much $\rightarrow$ less time to detect obstacles, increasing miss-detection risk (and collision risk).

1.3 Why Existing Approaches (Fixed Scheduling) Fail

Prior work in refs. [2]–[4] proposed **fixed or time-cycle-based** schedules:

- **IEEE 802.11ad (ref. [2]):** Reserve preamble blocks for radar, data blocks for communication within each 802.11ad frame. The split is fixed across all time cycles.
- **Static time-cycle sharing (ref. [4]):** Each “cycle” is divided into a fixed radar phase and a fixed communication phase — e.g., first 40% of each 100ms cycle for radar, 60% for communication.
- **Beam-alignment overhead reduction (ref. [3]):** Optimize V2I beam alignment, ignoring radar performance altogether.

The flaw: The environment is **dynamic and uncertain**. Weather changes minute by minute. Road conditions, AV speed, nearby traffic — these vary continuously. A fixed schedule can’t adapt:

- On a clear highway at low speed → radar detection risk is low; we could afford to use communication mode more often to transmit more data.
- In heavy rain at high speed on a congested urban road → unexpected obstacles are much more likely; we should use radar mode more aggressively, even if it cuts data throughput.

1.4 The Motivation for Learned, Adaptive Policies

Instead of a fixed schedule, the paper proposes an **adaptive, learned policy**: the AV observes its current state (weather, speed, queue state, channel state, etc.) and **learns** which mode choice is optimal at that state — maximizing long-term throughput while keeping miss-detection probability acceptable.

This requires:

1. A formal model of the problem (an MDP — Markov Decision Process).
2. A learning algorithm that finds the optimal policy despite **not knowing the environment’s transition probabilities in advance** (hence “deep reinforcement learning”).

That is the core innovation: not a new hardware design, but an intelligent software controller that makes mode decisions on the fly.

2 System Model: States, Modes, Metrics

Paper Reference: Section II (System Model)

2.1 The Data Queue and Communication Mode

The AV has a single communication channel to base stations. It maintains a **data queue** (a buffer) of incoming packets — e.g., sensor readings, video frames, traffic updates. The queue has a **maximum capacity** D (paper sets $D = 10$ packets).

Each time step (e.g., 1 second):

- New packets **arrive** at the queue (modeled as Poisson with rate $\lambda_d = 1$ packet/step).
- If the queue is full ($d = D$), new arrivals are dropped.
- If the AV chooses **communication mode** ($a = 0$):
 - It transmits some packets over the channel.
 - How many packets get through depends on the **channel state** $c \in \{0, 1\}$:
 - * $c = 0$ (good channel): transmit $\nu_1 = 4$ packets successfully.
 - * $c = 1$ (bad channel, high interference): transmit $\nu_2 = 2$ packets successfully.
 - The **channel switches** between good/bad randomly each step, with $P(\text{bad channel}) = p_c = 0.1$ (i.e., 90% of the time the channel is good).

Queue dynamics: $d_{t+1} = \min(d_t - (\text{transmission count if } a_t = 0) + (\text{arrivals}), D)$.

The **throughput metric** is the average number of packets successfully transmitted per time step over a long episode.

2.2 The Radar Mode and Unexpected Events

When the AV chooses **radar mode** ($a = 1$), it scans for **unexpected events** — an obstacle that could cause a collision:

- Outside the camera’s field of view.
- Obscured by weather or nearby vehicles.
- Example: a car turning from an intersecting road, obscured behind a parked truck (Fig. 1 in the paper).

Key assumption: the AV’s radar is **perfect** — if an unexpected event is actually present, the radar detects it with 100% probability. There’s no miss-detection or false alarm *within* the radar system itself; the risk is entirely about **when to use the radar** vs. communication.

(This simplification is worth noting: real radar has imperfect detection, especially in rain. A relaxation to add uncertainty in the radar itself is a potential future novelty direction.)

2.3 Four Environmental Factors and Their States

The **probability that an unexpected event occurs** in a given time step depends on four environmental factors, each binary:

Factor	Variable	Favorable (state 0)	Unfavorable (state 1)
Road condition	r	Straight, dry	Slippery, wet
Weather	w	Clear skies	Heavy rain
AV speed	v	Low (< 60 km/h)	High (≥ 60 km/h)
Nearby moving object	m	No	Yes (traffic nearby)

Each factor being unfavorable (state 1) increases the probability of an unexpected event. For instance:

- Wet roads reduce grip, increasing collision risk.
- Heavy rain reduces visibility, making obstacles harder to spot in advance.
- High speed means less time to react to obstacles.
- Nearby traffic means more potential for sudden movements.

How do these states evolve? The paper doesn't explicitly specify. The natural interpretation (and what we'll use) is that each factor is resampled independently each time step according to some underlying probability:

$$P(r_t = 1) = 1 - \tau_r \quad (\text{i.e., probability of unfavorable road at step } t) \quad (1)$$

$$P(w_t = 1) = 1 - \tau_w \quad (2)$$

$$P(v_t = 1) = 1 - \tau_v \quad (3)$$

$$P(m_t = 1) = 1 - \tau_m \quad (4)$$

(The paper provides $p_0^v, p_1^v, p_0^w, p_1^w$ from real-world data, but not the state-transition probabilities τ_i directly — this is a gap we'll note when implementing.)

2.4 The Dual Performance Metrics and the Tradeoff

The system has **two competing objectives**:

1. **Data Throughput:** average number of packets transmitted per step.
 - Maximize by choosing communication mode as often as possible.
2. **Miss-Detection Probability:** fraction of unexpected events the AV fails to detect.

- Minimize by choosing radar mode whenever events are likely.

The fundamental tradeoff: more radar $\rightarrow$ lower miss-detection but lower throughput.
More communication $\rightarrow$ higher throughput but higher miss-detection risk.

The paper's challenge: find a **single policy** that balances both objectives intelligently, adapting to the current environment state.

3 Environment Model and Event Probability (Equation 1)

Paper Reference: Section II (System Model) — Environment Dynamics

3.1 Bayesian Combination of Per-Factor Probabilities

Given the current state $(r, w, v, m) \in \{0, 1\}^4$, what's the probability that an unexpected event ($\oplus$) occurs this time step?

The paper defines:

- p_j^i : the conditional probability of an unexpected event given that factor i is in state j . For example:
 - $p_1^w = P(\oplus \mid w = 1)$ = probability of an obstacle given heavy rain.
 - $p_0^v = P(\oplus \mid v = 0)$ = probability of an obstacle given low speed.

The paper provides **from real-world data** (refs. [5], [6]):

$$p_0^v = 0.005, \quad p_1^v = 0.1 \quad (\text{high speed massively increases obstacle probability})$$

$$p_0^w = 0.005, \quad p_1^w = 0.046 \quad (\text{rain roughly } 10\times \text{ increases obstacle probability})$$

$$p_0^r, p_1^r \quad (\text{road condition}): \text{ not provided in the paper}$$

$$p_0^m, p_1^m \quad (\text{moving objects}): \text{ not provided in the paper}$$

To combine all four factors into a single $P(\oplus \mid r, w, v, m)$, the paper invokes Bayes' theorem and assumes **independence** of factors. The result (eq. 1):

$$P(\oplus) = \prod_{i \in \{r, w, v, m\}} [\tau_i p_0^i + (1 - \tau_i) p_1^i] \quad (5)$$

where $\tau_i \in [0, 1]$ is the probability that factor i is at the favorable state (0).

3.2 Concrete Example: Event Probability Calculation

Let's compute $P(\oplus)$ for a specific scenario:

- Road: dry ($r = 0$)
- Weather: rainy ($w = 1$)
- Speed: high ($v = 1$)
- Moving objects: yes ($m = 1$)

Using the paper's given values and reasonable placeholders:

$$\begin{aligned}\tau_r &= 0.7 \quad (70\% \text{ of time, road is dry}), & p_0^r &= 0.005, & p_1^r &= 0.05 \text{ (estimate)} \\ \tau_w &= 0.8 \quad (80\% \text{ clear weather}), & p_0^w &= 0.005, & p_1^w &= 0.046 \text{ (from paper)} \\ \tau_v &= 0.6 \quad (60\% \text{ low-speed driving}), & p_0^v &= 0.005, & p_1^v &= 0.1 \text{ (from paper)} \\ \tau_m &= 0.7 \quad (70\% \text{ no nearby objects}), & p_0^m &= 0.01, & p_1^m &= 0.08 \text{ (estimate)}\end{aligned}$$

For the specific state ($r = 0, w = 1, v = 1, m = 1$):

$$P(\oplus) = [0.7 \cdot 0.005 + 0.3 \cdot 0.05] \times [0.2 \cdot 0.005 + 0.8 \cdot 0.046] \times [0.4 \cdot 0.005 + 0.6 \cdot 0.1] \times [0.7 \cdot 0.01 + 0.3 \cdot 0.08] \quad (6)$$

$$P(\oplus) \approx 0.0185 \times 0.0369 \times 0.0602 \times 0.031 \approx 0.0000128 \quad (7)$$

Alternatively, if the road is slippery ($r = 1, w = 1, v = 1, m = 1$):

$$P(\oplus) \approx 0.041 \times 0.0369 \times 0.0602 \times 0.031 \approx 0.0000276 \quad (8)$$

Note: these are low absolute probabilities because most individual p_j^i values are small. But the **relative** ordering tells the story: as conditions worsen, $P(\oplus)$ increases, and that increasing risk should drive the policy toward radar mode.

3.3 Independence Assumption

The multiplicative form assumes the four factors are **independent** — the probability of an unexpected event given road + weather is the product of the separate probabilities. In reality, these factors are correlated (bad weather and slippery roads often co-occur). This simplification is worth flagging as a potential model extension later: a more realistic joint distribution of (r, w, v, m) might improve fidelity.

4 Problem Formulation: The Markov Decision Process (MDP)

Paper Reference: Section III (Problem Formulation)

4.1 The State Space S (Equation 2)

At each time step t , the AV observes a state $s_t \in S$. The state contains six pieces of information:

$$S = \{(d, c, r, w, v, m) : d \in \{0, 1, \dots, D\}, c \in \{0, 1\}, r, w, v, m \in \{0, 1\}\} \quad (9)$$

where:

- d = number of packets in the queue (0 to $D = 10$)
- c = channel state (0 = good, 1 = bad)
- r = road condition (0 = favorable, 1 = unfavorable)
- w = weather (0 = favorable, 1 = unfavorable)
- v = speed (0 = low, 1 = high)
- m = nearby moving objects (0 = none, 1 = present)

State space size: $11 \times 2 \times 2 \times 2 \times 2 \times 2 = 352$ possible states.

This is small enough for some classical RL algorithms (e.g., tabular Q-learning) to handle, but large enough that approximation methods (like neural networks) offer benefits in terms of learning speed and generalization.

4.2 The Action Space A

At each time step, the AV chooses exactly one action:

$$A = \{0, 1\} \quad (10)$$

where:

- $a = 0$: communicate mode (transmit packets).
- $a = 1$: radar mode (scan for obstacles).

4.3 The Reward Function (Equation 3)

After taking action a_t in state s_t , the AV receives an immediate reward r_t . The reward is designed to encode both objectives (throughput + safety) and guide learning toward a good policy.

4.3.1 Case 1: Communication mode ($a = 0$), no unexpected event ($\neg\oplus$)

- If channel is good ($c = 0$): $\mathbf{r}_t = +\mathbf{r}_1 = +2$
 - Intuition: successfully sent 4 packets ($\nu_1 = 4$), reward them.
- If channel is bad ($c = 1$): $\mathbf{r}_t = +\mathbf{r}_2 = +1$
 - Intuition: only 2 packets sent ($\nu_2 = 2$), lower reward.

4.3.2 Case 2: Communication mode ($a = 0$), but an unexpected event occurs ($\oplus$)

- $\mathbf{r}_t = -\mathbf{r}_3 = -50$
 - Intuition: catastrophic! The AV didn't use radar and missed an obstacle. Large negative penalty to strongly discourage this outcome.
 - Note: $r_3 = 50 \gg r_1, r_2$, which means the penalty for missing an event far outweighs the benefit of transmitting a few packets.

4.3.3 Case 3: Radar mode ($a = 1$), no unexpected event ($\neg\oplus$)

- $\mathbf{r}_t = 0$
 - Intuition: radar mode was used but there was no threat to detect. The radar provides value as a safety net, but using it wastefully (when there's nothing to detect) doesn't earn reward — it's the “cost” of safety.

4.3.4 Case 4: Radar mode ($a = 1$), and an unexpected event occurs ($\oplus$)

- $\mathbf{r}_t = +\mathbf{r}_4 \cdot (\mathbf{b} + 1) = +5 \cdot (\mathbf{b} + 1)$
 - where b = number of unfavorable factors among $\{r, w, v, m\}$.
 - Intuition: successfully detected an obstacle (radar works). Reward scales with how risky the environment was (b). If $b = 0$ (all factors favorable), minimal reward (+5). If $b = 4$ (all factors unfavorable, maximum danger), reward = +25.
 - This encourages the AV to use radar most aggressively in high-risk conditions and rewards actually preventing collisions in those conditions.

4.3.5 Summary Table

Action	Event?	Channel	Reward
Comm	No	Good	+2
Comm	No	Bad	+1
Comm	Yes	—	−50
Radar	No	—	0
Radar	Yes	—	$+5 \cdot (b + 1)$, where $b \in \{0, 1, 2, 3, 4\}$

The **asymmetry** is intentional: the penalty for missing an event (−50) is much larger than the rewards for successfully transmitting packets (+1, +2). This encodes a safety-first philosophy: don’t crash in pursuit of throughput.

4.4 The Objective: Discounted Return and Optimal Policy (Equation 4)

The AV’s goal is to find a **policy** π : a function that maps each state s to an action $\pi(s) \in A$.

Given a policy π , the expected outcome is the **discounted cumulative reward** (discounted return):

$$G(\pi) = E \left\{ \sum_{t=0}^T \gamma^t r_{t+1}(\pi) \right\} \quad (11)$$

where:

- $r_{t+1}(\pi)$ = immediate reward received at time step $t + 1$ under policy π .
- $\gamma \in (0, 1)$ = discount factor (paper doesn’t specify; typical value ≈ 0.99).
 - γ close to 1 means future rewards matter almost as much as immediate rewards.
 - γ close to 0 means the AV is myopic, only caring about immediate rewards.
- T = time horizon (episode length; paper doesn’t specify exactly, but simulation episodes appear to run for hundreds of steps).

The **optimal policy** π^* maximizes this expected return:

$$\pi^* = \arg \max_{\pi} G(\pi) \quad (12)$$

Intuitively: the optimal policy is the one that, when followed, leads to the highest expected cumulative reward over the long run. It balances immediate gains (transmitting packets) against long-term risks (missing obstacles).

4.5 Why This Is an MDP (and Why Reinforcement Learning Is Needed)

This problem fits the **Markov Decision Process** framework:

1. **States** are fully observable (the AV can measure d, c, r, w, v, m).
2. **Transitions** are Markovian: given the current state and action, the next state's distribution depends only on s_t and a_t , not the full history. (E.g., the queue evolution depends only on current d and the arrival/transmission this step.)
3. **Rewards** are deterministic given (s, a, s') .
4. **Discount**: future rewards are exponentially discounted.

In principle, one could solve this MDP **exactly** using dynamic programming if we knew the full transition probability matrix P (i.e., $P(s' | s, a)$ for all s, a, s'). But the paper doesn't assume we know P . Instead, it uses **deep reinforcement learning** to learn the optimal policy from **interaction** — the AV tries actions, observes rewards and next states, and gradually learns which actions are best in each state.

5 Bridge to the Next Sections (Not Covered Here)

***Paper Reference:** Sections IV & V (Deep Reinforcement Learning Algorithm and Performance Evaluation) — To be covered separately*

This document covers the problem setup and MDP formulation. The full paper continues with:

- **Section IV (Deep Reinforcement Learning Algorithm):** introduces the DQN algorithm (Deep Q-Network), which uses a neural network to approximate the optimal action-value function $Q^*(s, a)$ and thus find the optimal policy without knowing P .
- **Section V (Performance Evaluation):** describes the experimental setup, including the three baseline schemes (DQN, Q-learning, Round-robin), how they're trained, and the results comparing their convergence speed, throughput, and miss-detection probability across varying environmental conditions (swept p_1^v).

These will be covered in a separate study document once this foundation is solid.

6 Modeling Gaps and Questions for Implementation

Observation: *Across Sections I–III of the paper*

As noted above, several aspects of the system model are underspecified in the paper and will need explicit choices during implementation:

1. **Missing probability values:** $p_0^r, p_1^r, p_0^m, p_1^m$ are not given. Reasonable estimates will be chosen and documented.
2. **State evolution:** How do r, w, v, m evolve over time? The paper suggests each is resampled per step, but τ_i (state-transition probabilities) are not given. Again, reasonable defaults will be chosen.
3. **Exact event-generation process:** Equation 1 gives $P(\oplus)$ as a marginal probability. The paper doesn't explicitly state whether each step's event ($\oplus$ or $\neg\oplus$) is a Bernoulli draw with this probability. We'll assume yes, but this is an assumption.
4. **Discount factor γ and episode length T :** Not specified in the paper. Typical values will be used ($\gamma \approx 0.99$, $T \approx 500$ –1000 steps per episode).
5. **DQN hyperparameters:** learning rate, batch size, replay buffer size, network architecture, ε -decay schedule — none of these are fully specified in the paper's 4-page format. Classical DQN choices will be documented.

All of these choices will be made explicit and justified during the implementation phase, so that reproduction results are reproducible and deviations from the paper's reported results can be traced to specific modeling decisions, not mystery.

Last Updated: August 23, 2026 **Status:** Deep-understanding phase (Sections I–III)

Next: Study of Sections IV–V (DQN algorithm and experimental results) — separate document.